

Bug 1: Search and Status filter

Where I found it:

The issue was in TaskRepository.java. The same SQL problem was also present in db/queries/search_tasks.sql and the Oracle package.

How I found it:

When I searched for api, archived tasks were also appearing. The Done filter was also not working correctly.

Examples:

- q=api returned archived tasks.
- status=DONE returned more tasks than expected.
- Searching with an empty query made the status filter ineffective.

Root Cause:

The SQL condition used AND and OR without proper ~~brackets~~ brackets.

The original logic was effectively:

A AND B OR C AND D

SQL evaluates ~~OR~~ AND before OR, so the conditions were not grouped as intended.

Also, when search was empty, LIKE '%' matched everything, which made the status condition ineffective for some rows.

fix:

I grouped the search conditions using ~~parenthesis~~ parentheses:

```

{ archived = FALSE
  AND (title LIKE ... OR description LIKE ...)
  AND (status IS NULL OR status = ?)

```

I applied the same correction to all SQL copies.

Why this fix:

This was the smallest and safest change. It makes the intended logic explicit and prevents archived or incorrect-status tasks from being returned.

Result:

- ✓ Search works correctly.
- ✓ Done filter works correctly.
- ✓ Archived tasks are excluded.
- ✓ Search and status can now be used together.

Bug 2: Stale Search Results

Where I found it:

The issue was in:

frontend/src/hooks/useTasks.js

It was made easier to reproduce because the backend had an artificial delay in TaskController.java.

How I found it:

When typing quickly, the browser sent multiple requests:

a → ap → api

The older request could finish after the newer request and overwrite the latest result.

Root Cause:

There was no ~~cancellation~~ cancellation of previous requests.

The backend delay made shorter/older queries slower, which increased the chance of an old response arriving after the new ~~response~~ response.

fix:

I used AbortController in the React effect.

For every new request:

1. Create a new AbortController.
2. Pass its signal to fetch.
3. Abort the previous request during effect cleanup.
4. Ignore AbortError.

I also removed the artificial backend sleep.

Why this fix:

Removing the backend delay alone would only hide the problem. The real fix is cancelling requests that are no longer needed.

This also saves ~~unnecessary~~ unnecessary network and server work.

Important concept:

React useEffect cleanup runs before the next effect and when the component is unmounted.

Result:

✓ Old search requests cannot overwrite the latest search result.

Bug 3: Loading State and Error HandlingWhere:

frontend/src/hooks/useTasks.js

How I found it:

When an API request failed, the error message was set, but the loading state could remain true. Also, an old error could remain visible when a new request started.

Root Cause:

The error path did not correctly reset loading, and the previous error was not cleared at the ~~beginning~~ beginning of a new request.

Fix:

At the beginning of a request:

setError(null)

On ~~success~~ successful completion:

setLoading(false)

On a real error:

setError(...)

setLoading(false)

I did not treat an AbortError as a real error because it means an older request was intentionally cancelled.

Why:

The UI should represent the state of the current request, not an older request.

Bug 4: Pagination Does Not ResetWhere:

frontend/src/App.js

How I found it:

If I went to page 4 and then performed a new search, the new result set might only have one page.

The application could remain on page 4 and show no tasks.

Root Cause:

The search/status value changed, but the current page number stayed ~~same~~ unchanged.

Fix:

When search changes:

setQuery(value)

setPage(1)

When status changes:

setStatus(value)

setPage(1)

Why:

A new search or filter creates a new result set, so pagination should always start from page 1.

I reset the page in the same event handler instead of using another effect, because that avoids an unnecessary request for the old page.

Result:

✓ Search and status filtering now correctly return to page 1.

Bug 5: Invalid API Input Causes 500 ErrorWhere:

backend/src/main/java/com/internal/task tracker/TaskController.java

How I found it:

Invalid inputs such as: status=BOGUS or page=0 could cause server errors instead of returning a proper client error.

A very large pageSize could also request an unreasonable amount of data.

Root Cause:

The controller did not validate the incoming ~~parameters~~ parameters before using them.

TaskStatus.valueOf() can throw an exception for an invalid status.

Invalid page values can also cause incorrect offsets or list operations.

fix:

I added validation:

- Page must be at least 1.
- Page size must be between 1 and 100.
- Status must be one of the allowed values.
- Invalid input returns HTTP ~~404~~ 400 with an error message.
- Offset calculation uses a long to avoid integer overflow.

I also updated the frontend API code to display the server's error message.

Why 400 instead of silently fixing the value:

HTTP 400 clearly tells the client that its request is invalid. Silently changing the value could hide bugs in the client.

Bug 6: Too Many Search RequestsWhere:

frontend/src/components/SearchBar.jsx

How I found it:

Typing api originally caused approximately three API requests: a, ap, api

fix:

I added a 300 ms debounce.

The search input keeps its own local value and waits until the user stops typing before calling the parent search handler.

The previous timeout is cleared when the user types again.

Why:

This reduces unnecessary API calls while keeping the search ~~responsive~~ responsive.

Other improvements I reviewed/fixed:

- Escaped SQL LIKE wildcard characters % and _.
- Added ESCAPE '!' for correct literal wildcard searching.
- Added stable ordering using created_at DESC, id DESC.
- Replaced System.out.println with logging.
- Added aria-label attributes for ~~accessibility~~ accessibility.
- Applied the SQL precedence fix to the Oracle package too.
- Added Oracle input validation for page and status values.

final verification: I verified that -

- ✓ Search works.
- ✓ Done after works.
- ✓ Invalid status returns a proper 400 error instead of crashing.
- ✓ State search results are prevented.
- ✓ Loading and error states are handled correctly.
- ✓ Pagination resets when search/filter changes.